



CHAPTER 01.1: BIG-O & ALGORITHM FUNDAMENTALS

PAGE 1 OF 6

1 WHAT IS AN ALGORITHM?

An **algorithm** is a deterministic, step-by-step procedure for solving a computational problem in a finite amount of time.

Input Data → [Systematic Processing] → Output Result

2 WHY ALGORITHMS MATTER

⚡ **Hardware Scale Limits:** CPU speed cannot fix an inefficient $O(N^2)$ algorithm.

💰 **Cloud Server Cost:** $O(N^2)$ on 10M requests costs thousands in server bills.

👛 **FAANG Engineering Standard:** Designing systems for 10M+ req/sec.

3 REAL-WORLD ANALOGY: SEARCHING A DICTIONARY

Linear Search (Page-by-Page):

Flipping through a 1,000-page dictionary page by page from start to finish. If the word is at the end, you perform 1,000 checks. $O(N)$ Complexity.

Binary Search (Halving):

Opening the exact middle page (page 500), checking the letter, and eliminating 500 pages instantly. Repeats in only 10 steps! $O(\log N)$ Complexity.

CHAPTER 01.1 LEARNING ROADMAP

Pages 1–2: Algorithm Definitions & Time vs Space Tradeoffs

Pages 3–4: Asymptotic Notation (O , Ω , Θ) & Code Analysis Rules

Pages 5–6: Call Stack Memory & Master Complexity Cheat Sheet

🚀 **KEY TAKEAWAY:** Algorithms transform brute-force computation into scalable systems. Turn the page to explore Time vs Space Complexity!



1 TIME COMPLEXITY

Measures how the number of **CPU operations** grows as input size N scales towards infinity.

Not measured in seconds (varies across hardware & CPU load).

2 SPACE COMPLEXITY

Measures how much **extra RAM memory** is allocated by the algorithm as input size N scales.

Auxiliary Space (extra algorithm memory) vs Total Space.

3 WHY FAANG INTERVIEWERS CARE

1. System Scale Failure: An $O(N^2)$ algorithm works for 100 items but crashes production servers for 1,000,000 items.

2. Memory Footprint: Unnecessary $O(N)$ memory allocations in microservices trigger OutOfMemory (OOM) errors under load.



TIME VS SPACE TRADE-OFF MATRIX

Dimension	Time Complexity	Space Complexity
What it tracks	Number of elementary CPU instructions executed	Auxiliary RAM memory slots allocated
Hardware Limiter	CPU clock speed & thread execution cores	Heap RAM & Call Stack memory limits
Classic Trade-off	Use HashMaps (Space) to reduce search time from $O(N)$ to $O(1)$	Use In-Place Swapping (Time) to eliminate extra array allocation



INTERVIEW TIP: Always state both Time AND Space complexity immediately after proposing a solution!

BIG-O (O)

Upper Bound (Worst Case):

Guarantees algorithm will run *no worse than* $f(N)$. Primary focus of FAANG interviews!

BIG-THETA (Θ)

Tight Bound (Average Case):

Exact mathematical growth rate when upper and lower bounds match.

BIG-OMEGA (Ω)

Lower Bound (Best Case):

Guarantees algorithm will run *at least as fast as* $f(N)$.



COMPLEXITY GROWTH RATES (SLOWEST TO FASTEST)

1. **Constant $O(1)$:** Hash Table Lookups, Array Indexing

⚡ EXCELLENT

2. **Logarithmic $O(\log N)$:** Binary Search, Balanced BST Lookup

✓ GREAT

3. **Linear $O(N)$:** Single Loop Traversal, Array Search

👍 GOOD

4. **Linearithmic $O(N \log N)$:** Merge Sort, Quick Sort

⚠ FAIR

5. **Quadratic $O(N^2)$:** Nested Loops, Bubble Sort

🐢 SLOW

6. **Exponential $O(2^n)$:** All Subsets Recursion, Fibonacci Brute

💣 BAD



INPUT SIZE VS OPERATION COUNT COMPARISON

Complexity	N = 10	N = 100	N = 1,000	N = 1,000,000
$O(1)$ Constant	1	1	1	1
$O(\log N)$ Logarithmic	~3	~7	~10	~20
$O(N)$ Linear	10	100	1,000	1,000,000
$O(N \log N)$ Linearithmic	~33	~664	~9,965	~ 20×10^6
$O(N^2)$ Quadratic	100	10,000	1,000,000	10^{12} (CRASH)

💡 **AHA MOMENT:** $O(\log N)$ is practically constant time! For $N = 1,000,000,000$ (1 billion items), binary search takes only 30 operations!



HOW TO CALCULATE TIME COMPLEXITY (CODE DRILLS)

PAGE 4 OF 6

1. SINGLE LOOP → O(N)

```
for (int i = 0; i < n; i++) {  
    // 1 operation per iteration  
}  
// Total = N iterations → O(N) Time
```

2. NESTED LOOP → O(N²)

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        // N * N operations  
    }  
} // Total = N^2 iterations →  
O(N^2) Time
```

3. LOGARITHMIC LOOP → O(log N)

```
for (int i = 1; i < n; i *= 2) {  
    // i doubles each step: 1, 2, 4,  
    8 ...  
} // Takes log2(N) steps → O(log N)  
Time
```

4. INDEPENDENT INPUTS → O(A + B)

```
for (int x : arrA) { /* A times */ }  
for (int y : arrB) { /* B times */ }  
// Two different arrays → O(A + B)  
Time
```

4 GOLDEN RULES OF CODE COMPLEXITY ANALYSIS

Rule	Description	Example Transformation
1. Drop Constants	Ignore multiplier constants (CPU scale factors)	$O(2N + 5) \rightarrow O(N)$
2. Drop Non-Dominant Terms	Keep only the highest order growth term	$O(N^2 + N + 100) \rightarrow O(N^2)$
3. Sequential Add, Nested Multiply	Loops side-by-side add; loops inside loops multiply	Loop A + Loop B = $O(A+B)$
4. Halving / Doubling = Log N	Any loop dividing search space by factor k	Binary Search / Tree Height = $O(\log N)$

⚠ **COMMON TRAP:** Calling `String.indexOf()` or `list.contains()` inside a loop creates a hidden nested $O(N^2)$ loop!



1 PRIMITIVE VARIABLES → O(1)

```
int sum = 0;
int left = 0, right = n - 1;
// Fixed number of variables → O(1)
Space
```

2 ARRAYS & HASHMAPS → O(N)

```
int[] copy = new int[n];
Set<Integer> set = new HashSet<>();
// Memory grows with N → O(N) Space
```

3 RECURSION CALL STACK MEMORY

Every recursive function call creates a **Stack Frame** in memory storing local variables and return addresses.

Balanced Tree Recursion: Stack depth equals height of tree = $O(\log N)$ Space.

Linear Recursion (Skewed): Stack depth equals N calls = $O(N)$ Space (StackOverflow risk).



HEAP MEMORY VS CALL STACK MEMORY

Memory Region	What is Stored	Space Impact
Call Stack	Primitive local variables & function return frames	Determined by maximum recursion depth
Heap Memory	Objects allocated with <code>new</code> (Arrays, HashMaps, Trees)	Determined by total count of live objects
In-Place Optimization	Modifying input array directly without <code>new</code> allocations	Reduces auxiliary space to O(1)

💡 **MEMORY BOX:** Auxiliary Space excludes input data itself! If a function accepts an array of size N and returns a scalar, auxiliary space is O(1).



★ DATA STRUCTURE & ALGORITHM COMPLEXITY CHEAT SHEET

Data Structure / Algorithm	Access / Search	Insert / Delete	Space Complexity
Array / ArrayList	Access: $O(1)$, Search: $O(N)$	$O(N)$ (shift elements)	$O(N)$
HashMap / HashSet	Access: $O(1)$, Search: $O(1)$	$O(1)$ average	$O(N)$
Binary Search Tree (Balanced)	Search: $O(\log N)$	$O(\log N)$	$O(N)$
Binary Search Algorithm	Search: $O(\log N)$	N/A (Sorted Array)	$O(1)$ iterative
Merge Sort / Quick Sort	Time: $O(N \log N)$	N/A	Merge: $O(N)$, Quick: $O(\log N)$

🎯 INPUT SIZE (N) → REQUIRED COMPLEXITY TRICK

$N \leq 10-12$:

Target $O(N!)$ or $O(2^N)$ Backtracking

$N \leq 5,000$:

Target $O(N^2)$ DP or Nested Loops

$N \leq 10^5$:

Target $O(N \log N)$ Sort or $O(N)$ Map

$N \geq 10^9$:

Target $O(\log N)$ or $O(1)$ Math/BS

🌸 LEETCODE PRACTICE DRILLS

- **LC 217 Contains Duplicate:** Compare $O(N^2)$ vs $O(N)$ HashSet
- **LC 242 Valid Anagram:** $O(N)$ Hashing vs $O(N \log N)$ Sorting
- **LC 704 Binary Search:** $O(\log N)$ Time, $O(1)$ Space
- **LC 53 Maximum Subarray:** Kadane's $O(N)$ vs $O(N^2)$ Brute

📌 20-SECOND REVISION SUMMARY

1. Big-O = Upper Bound (Worst Case). Focus on highest growth term.
2. Drop constants & non-dominant terms ($O(2N + 5) \rightarrow O(N)$).
3. Halving search space = $O(\log N)$. Nested loops = $O(N^2)$.
4. Auxiliary Space = extra memory allocated (excluding input).

✅ **CHAPTER 01.1 COMPLETE:** You now possess FAANG-level Big-O analysis skills! Next Chapter: **Chapter 01.2: Bit Manipulation & Math Tricks!**